

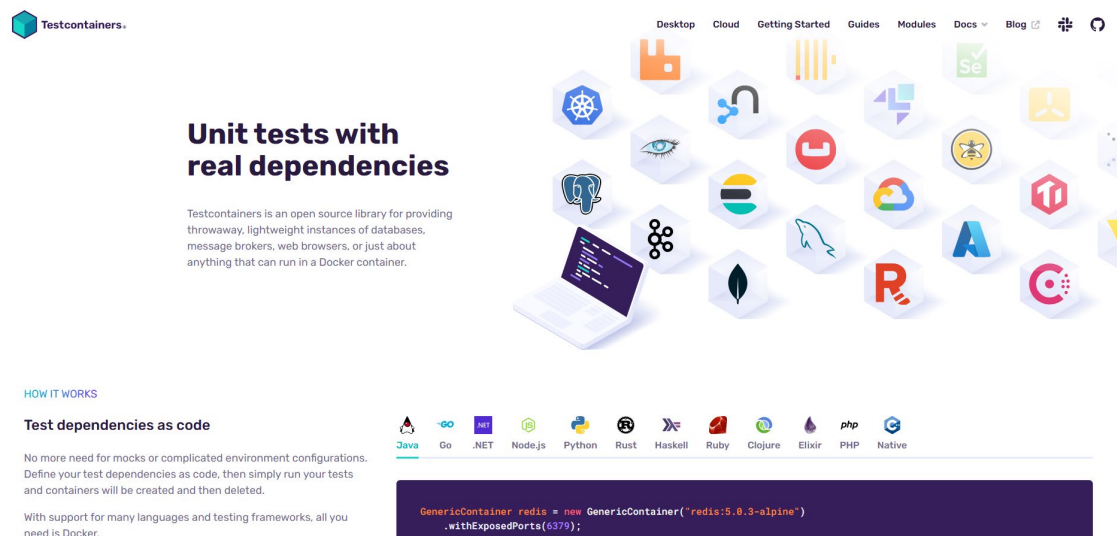
1 - Integration tests

Integration testing verifies that different parts of an application work together correctly, often involving real external dependencies like databases or message brokers. Unlike unit tests, integration tests check the interaction between components in a realistic environment.

Testcontainers is a Java library that helps create lightweight, throwaway instances of external services (like databases) using Docker containers during tests. This ensures tests run against real, isolated environments without manual setup, improving reliability and consistency.

Using integration tests with Testcontainers allows developers to automate end-to-end testing with real infrastructure components, leading to more robust and maintainable applications.

<https://testcontainers.com/>



The screenshot shows the Testcontainers website. At the top, there's a navigation bar with links: Desktop, Cloud, Getting Started, Guides, Modules, Docs, Blog, and icons for GitHub and Twitter. The main heading is "Unit tests with real dependencies". Below it, a subheading states: "Testcontainers is an open source library for providing throwaway, lightweight instances of databases, message brokers, web browsers, or just about anything that can run in a Docker container." To the right, there's a grid of icons representing various services like Redis, PostgreSQL, RabbitMQ, etc. Below the main heading, there's a section titled "HOW IT WORKS" with the subheading "Test dependencies as code". It explains that no more need for mocks or complicated environment configurations, and that test dependencies are defined as code. It also mentions support for many languages and testing frameworks, all requiring Docker. At the bottom, there's a code snippet in a dark box showing how to create a Redis container in Java using Testcontainers.

```
GenericContainer redis = new GenericContainer("redis:5.0.3-alpine")
    .withExposedPorts(6379);
```

Add dependencies to pom.xml to allow integration tests with test container.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-testcontainers</artifactId>
  <scope>test</scope>
</dependency>
```

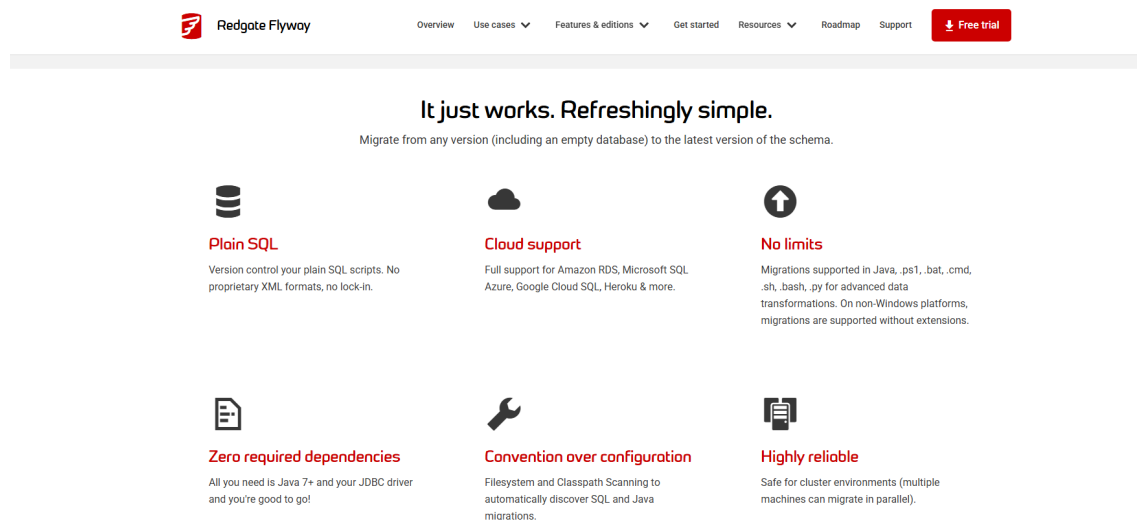
Add dependencies to allow rest communication during tests.

```
<dependency>
```

```
    <groupId>io.rest-assured</groupId>  
    <artifactId>rest-assured</artifactId>  
</dependency>
```

2 - Flyway

<https://www.red-gate.com/products/flyway/community/>



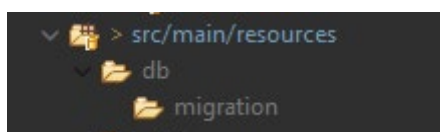
Flyway is a lightweight and powerful database migration tool. It helps manage and automate version control for your database schema. Flyway uses simple SQL or Java-based migration scripts to apply incremental changes to your database in a controlled and consistent way.

It integrates easily with Spring Boot and supports many relational databases like MySQL, PostgreSQL, Oracle, and more. With Flyway, you can ensure your database structure stays in sync across environments (development, testing, production) and team members.

Add the dependencies to enable flyway db migration

```
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-core</artifactId>
</dependency>
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-mysql</artifactId>
</dependency>
```

Whenever flyway is activated, it searches for the migration scripts in the next folder.



The scripts for migrations must follow the next name convention:

V<number>__<name>.sql → Examples: V1__orders.sql, V1__init.sql

3 - Spring Cloud Open Feign



Spring Cloud OpenFeign is a declarative HTTP client that makes it easy to call other microservices in a Spring Boot application.

◆ **What it does:**

It allows you to define Java interfaces to call REST APIs, avoiding boilerplate code with RestTemplate or WebClient.

◆ **How it works:**

Add the dependency:

spring-cloud-starter-openfeign

Enable Feign clients in your main class with @EnableFeignClients.

Create an interface with @FeignClient, specify the service name or URL, and define the endpoints using Spring annotations like @GetMapping.

Inject and use the interface as a regular Spring bean.

✓ Benefits:

- Simple and readable code
- Built-in integration with Spring Boot
- Supports load balancing, fallback, logging, and more

4 - WireMock

WireMock is a tool used to mock HTTP services, allowing you to simulate external APIs and test your application without depending on real remote services.

1. Add the WireMock dependency to your project (order-related module) (spring-cloud-starter-contract-stub-runner)
2. Use the `@AutoConfigureWireMock(port = ...)` annotation to configure WireMock on a specified port for integration testing

```
<!--  
    Enables the use of stubbed responses from external services during tests.  
    Useful when your app depends on other microservices – you can simulate their responses  
    without having to run the actual service.  
    Internally uses WireMock to spin up a fake HTTP server during tests.  
-->  
<dependency>  
    <groupId>org.springframework.cloud</groupId>  
    <artifactId>spring-cloud-starter-contract-stub-runner</artifactId>  
    <scope>test</scope>  
</dependency>  
<!--  
    Library for testing REST APIs in a fluent and readable way.  
    Allows you to easily send HTTP requests (GET, POST, etc.) and assert status, headers, and body.  
-->  
<dependency>  
    <groupId>io.rest-assured</groupId>  
    <artifactId>rest-assured</artifactId>  
</dependency>  
<!-- Open feign dependency to allow communication between microservices -->  
<dependency>  
    <groupId>org.springframework.cloud</groupId>  
    <artifactId>spring-cloud-starter-openfeign</artifactId>  
</dependency>
```

5 - API Gateway

An API Gateway is a server that acts as a single entry point for all client requests to a system made up of multiple services (usually microservices). It receives the requests, forwards them to the correct service, and returns the response back to the client.

Dependency:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway-server-webmvc</artifactId>
</dependency>
```

✓ Benefits:

Centralized entry point – Simplifies how clients access the system.

Security and monitoring – Easier to implement authentication, rate limiting, and logging.

Request routing – Routes traffic to the right service based on URL, method, etc.

Reduces client complexity – Clients don't need to know about service locations.

Request aggregation – Combines multiple service calls into one response.

✗ Drawbacks:

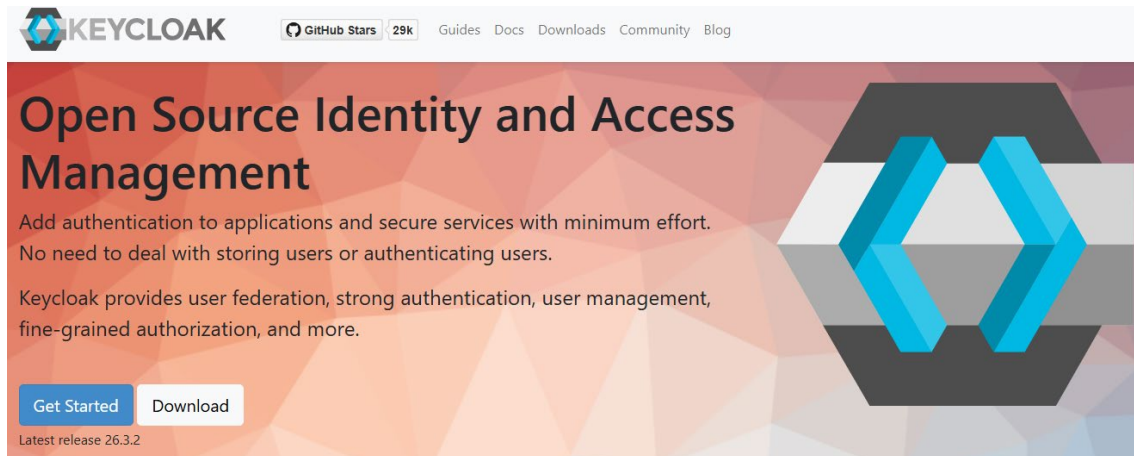
Single point of failure – If the gateway goes down, the whole system may become unreachable.

Increased latency – Adds an extra network hop between the client and services.

Complex configuration – Routing rules and policies can grow complex over time.

Scalability needed – The gateway must handle all traffic, so it must be well-scaled.

6 - KeyCloak

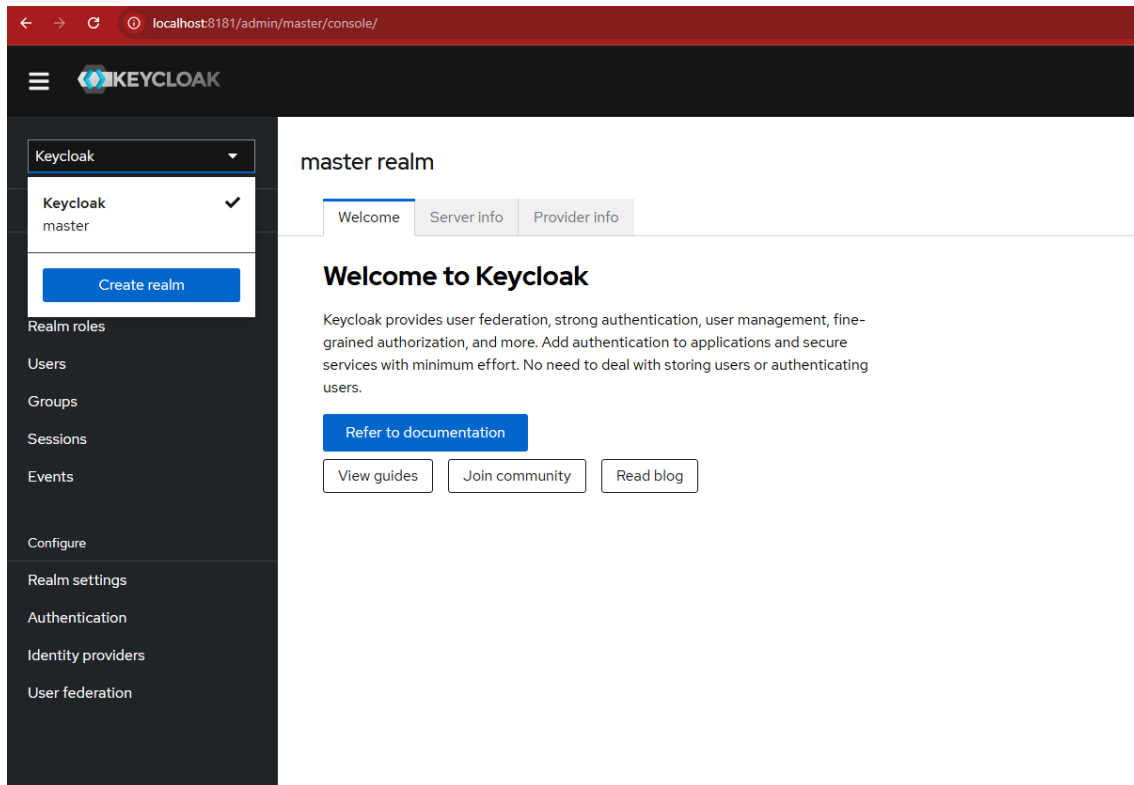


Keycloak is an open-source Identity and Access Management (IAM) solution developed by Red Hat. It provides features such as Single Sign-On (SSO), user federation, identity brokering, and centralized authentication and authorization for applications and services. With support for standard protocols like OAuth 2.0, OpenID Connect, and SAML 2.0, Keycloak enables secure and flexible user management out of the box. It also includes an easy-to-use admin console and integrates well with modern applications and microservice architectures.

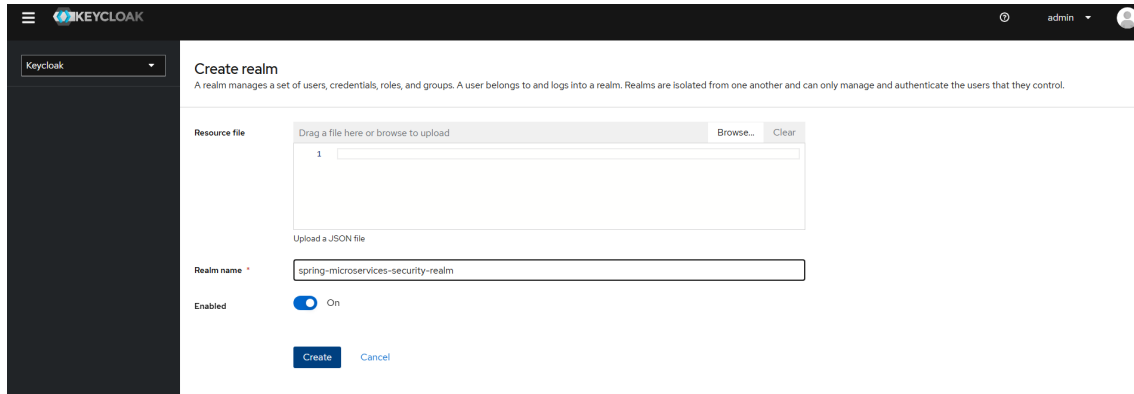
OAuth 2.0: It is an authorization protocol that allows applications to access a user's resources on another service (such as Google or Facebook) without needing to share the user's password.

OpenID Connect (OIDC): It is an identity layer built on top of OAuth 2.0. It allows applications to verify the user's identity and securely obtain basic profile information.

SAML (Security Assertion Markup Language): It is an XML-based authentication protocol used for Single Sign-On (SSO), especially in enterprise environments. It allows users to authenticate once and access multiple applications.



The first step is to create a new realm for the project configuration (users, roles, credentials etc).



The next step is to create a new client

localhost:8101/admin/master/console/#/spring-microservices-security-realm/clients/add-client

KEYCLOAK

spring-microservices-se...

Manage

Clients

Client scopes

Realm roles

Users

Groups

Sessions

Events

Configure

Realm settings

Authentication

Identity providers

User federation

Clients > Create client

Create client

Clients are applications and services that can request authentication of a user.

- General settings
- Capability config
- Login settings

Client type OpenID Connect

Client ID * spring-client-credentials-id

Name

Description

Always display in UI ☐ Off

Next Back Cancel

KEYCLOAK

spring-microservices-se...

Manage

Clients

Client scopes

Realm roles

Users

Groups

Sessions

Events

Configure

Realm settings

Authentication

Identity providers

User federation

Clients > Create client

Create client

Clients are applications and services that can request authentication of a user.

- General settings
- Capability config
- Login settings

Client authentication ☒ On

Authorization ☐ Off

Authentication flow ☐ Standard flow ☐ Implicit flow ☐ OAuth 2.0 Device Authorization Grant ☐ OIDC CIBA Grant

☐ Direct access grants ☒ Service accounts roles

Next Back Cancel

KEYCLOAK

spring-microservices-se...

Manage

Clients

Client scopes

Realm roles

Users

Groups

Sessions

Events

Configure

Realm settings

Authentication

Identity providers

User federation

Clients > Create client

Create client

Clients are applications and services that can request authentication of a user.

- General settings
- Capability config
- Login settings

Root URL

Home URL

Leave it empty

Save Back Cancel

Next, check the client secret

The screenshot shows the Keycloak administration console. On the left is a sidebar with navigation links: Manage, Clients, Client scopes, Realm roles, Users, Groups, Sessions, Events, Configure, Realm settings, Authentication, Identity providers, and User federation. The 'Clients' link is selected. The main content area shows the 'Client details' for 'spring-client-credentials-id' (OpenID Connect). Below the client name are tabs: Settings, Keys, Credentials (active), Roles, Client scopes, Service accounts roles, Sessions, and Advanced. The 'Credentials' tab contains three sections: 'Client Authenticator' (set to 'Client Id and Secret' with a 'Save' button), 'Client Secret' (a masked field with 'Regenerate' and a copy icon), and 'Registration access token' (a masked field with 'Regenerate' and a copy icon).

Through realm settings, we can access openId configuration endpoint

The screenshot shows the Keycloak administration console with the 'Realm settings' page for 'spring-microservices-security-realm'. The sidebar is the same as in the previous image, but 'Realm settings' is now selected. The main content area has tabs: General (active), Login, Email, Themes, Keys, Events, Localization, Security defenses, Sessions, Tokens, Client policies, User profile, and User registration. The 'General' tab contains fields for 'Realm ID' (spring-microservices-security-realm), 'Display name', 'HTML Display name', 'Frontend URL', and 'Require SSL' (set to 'External requests'). Below these are sections for 'ACR to LoA Mapping' (with a message and an 'Add ACR to LoA Mapping' link), 'User-managed access' (set to 'Off'), 'Unmanaged Attributes' (set to 'Disabled'), and 'Endpoints' (with links for 'OpenID Endpoint Configuration' and 'SAML 2.0 Identity Provider Metadata').

```
localhost:8181/realms/spring-microservices-security-realm/well-known/openid-configuration
{
  "issuer": "http://localhost:8181/realms/spring-microservices-security-realm",
  "authorization_endpoint": "http://localhost:8181/realms/spring-microservices-security-realm/protocol/openid-connect/auth",
  "token_endpoint": "http://localhost:8181/realms/spring-microservices-security-realm/protocol/openid-connect/token",
  "introspection_endpoint": "http://localhost:8181/realms/spring-microservices-security-realm/protocol/openid-connect/token/introspect",
  "userinfo_endpoint": "http://localhost:8181/realms/spring-microservices-security-realm/protocol/openid-connect/userinfo",
  "end_session_endpoint": "http://localhost:8181/realms/spring-microservices-security-realm/protocol/openid-connect/logout",
  "frontchannel_logout_session_supported": true,
  "frontchannel_logout_supported": true,
  "jwks_uri": "http://localhost:8181/realms/spring-microservices-security-realm/protocol/openid-connect/certs",
  "check_session_iframe": "http://localhost:8181/realms/spring-microservices-security-realm/protocol/openid-connect/login-status-iframe.html",
  "grant_types_supported": [
    "authorization_code",
    "implicit",
    "refresh_token",
    "password",
    "client_credentials",
    "urn:openid:params:grant-type:ciba",
    "urn:ietf:params:oauth:grant-type:device_code"
  ],
  "acr_values_supported": [
    "0",
    "1"
  ],
  "response_types_supported": [
    "code",
    "none",
    "id_token",
    "token",
    "id_token token",
    "code id_token",
    "code token",
    "code id_token token"
  ],
  "subject_types_supported": [
    "public",
    "pairwise"
  ],
  "id_token_signing_alg_values_supported": [
    "PS384",
    "RS384",
    "EdDSA",
    "ES384",
    "HS256",
    "HS512",
    "ES256",
    "RS256",
    "HS384"
  ]
}
```

The API Gateway module will be configured as a resource server for oauth2.
Add the next dependency:

```
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-oauth2-resource-server</artifactId>
</dependency>
```

In the openid endpoint (keycloak / realm settings), we can check the token url

```
{
  "issuer": "http://localhost:8181/realms/spring-microservices-security-realm",
  "authorization_endpoint": "http://localhost:8181/realms/spring-microservices-security-realm/protocol/openid-connect/auth",
  "token_endpoint": "http://localhost:8181/realms/spring-microservices-security-realm/protocol/openid-connect/token",
}
```

We can configure a new token for our requests in postman, so that we get authorized to access the endpoints of our application.
Besides the token url, we also need the client's ID and secret.

GET

http://localhost:9000/api/product

Params

Authorization

Headers (10)

Body

Scripts

Tests

Settings

Auth Type

OAuth 2.0

The authorization data will be automatically generated when you send the request. Learn more about [OAuth 2.0](#) authorization.

Add authorization data to

Request Headers

This will allow anyone with access to this request to view and use it.

Configure New Token

Token Name

Token

Grant type

Client Credentials

Access Token URL

http://localhost:8181/realms/spring-microse...

Client ID

.....

Client Secret

.....

Scope

e.g. read:org

Body

Cookies (1)

Headers (11)

Test Results

Raw

Preview

Visualize

1

Finally scroll to the bottom and clic the button to generate the token, then clic on proceed.

Refresh Request

	Key	Value	Se
	Create parameter	Value	

Clear cookies

Get New Access Token

Get new access token

Authentication complete

This dialog box will automatically close in 2...

Proceed

MANAGE ACCESS TOKENS

All TokensDelete ▼Token DetailsUse Token

TokenToken NameToken Access Token

Token eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUiwiaw2lkIA6ICjNeHQwYzBFS1dZMk5UMEETbGVGMik2T3RocEI5aFlwWHJEctThuemRTnRln0.eyJleHAiOjE3NTQ0ODkwNTYsImhhdCI6MTc1NDQ4ODc1NiwiianRpIjoimNm4MmY4ZjMtZjgxZC00OGM0LTgwM2UtYWU1Mja5YzJjMTFkliwiaXNlZjoiaHR0cDovL2xvY2FsaG9zdDo4MTgxL3JlYWxtcy9zcHJpbmctbWljcm9zZXJ2aWNlc y1zZW N1cm l0eS1yZW FsbSIslmF1ZCI6ImFjY291bnQiLCJzdWllOiIlyOTkwZ DQwMC1iy2EyLTQ4YjEtOTBjMCM1hyjk1ZDM1NDJhODUiLCJ0eXAiOiJCZ WfY2XliLCJhenAiOiJzcHJpbmctY2xpZW50LWNyZWRIbnRpYWxzLWlkli wiYNWnljoiMSlslmFsbG93ZWQt b3JpZ2lucyl6WylvKiJdLCJyZW FsbV9hY 2Nlc3MiOnsicm9sZXMiOiIsib2ZmbGluZV9hY2Nlc3MiLCJkZWZh dWx0LX JvbGVzLXNWcmcluzY1taWNyb3NlcnZpY2VzLXNIY3VyaXR5LXIyYXtliwi dW1hX2F1dGhvcmcl6YXRpb24iXX0sInJlc291cmNIX2FjY2Vzcyl6eyJhY2N

Create client

Clients are applications and services that can request authentication of a user.

1 General settings

2 Capability config

3 Login settings

Client authentication ⓘ
☐ Off

Authorization ⓘ
☐ Off

Authentication flow
☒ Standard flow ⓘ
☐ Implicit flow ⓘ
☐ OAuth 2.0 Device Authorization Grant ⓘ
☐ OIDC CIBA Grant ⓘ
☒ Direct access grants ⓘ
☐ Service accounts roles ⓘ

Next

Back

Cancel

Valid redirect URIs ⓘ

http://localhost:4200

+

Add valid redirect URIs

Valid post logout redirect URIs ⓘ

+

Add valid post logout redirect URIs

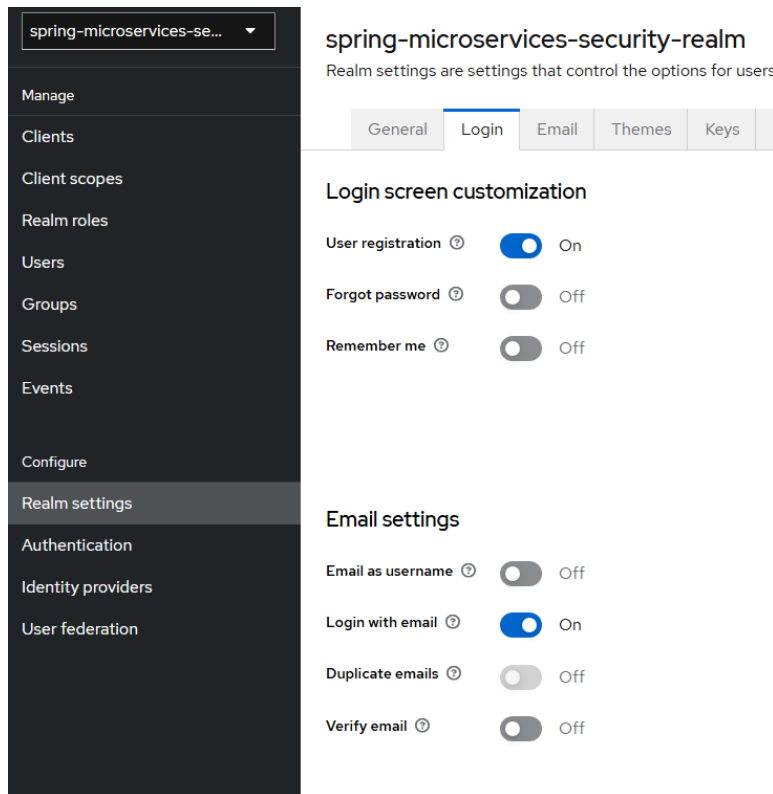
Web origins ⓘ

*

+

Add web origins

We can also activate the user registration option in our realm settings.



7 - Circuit breaker



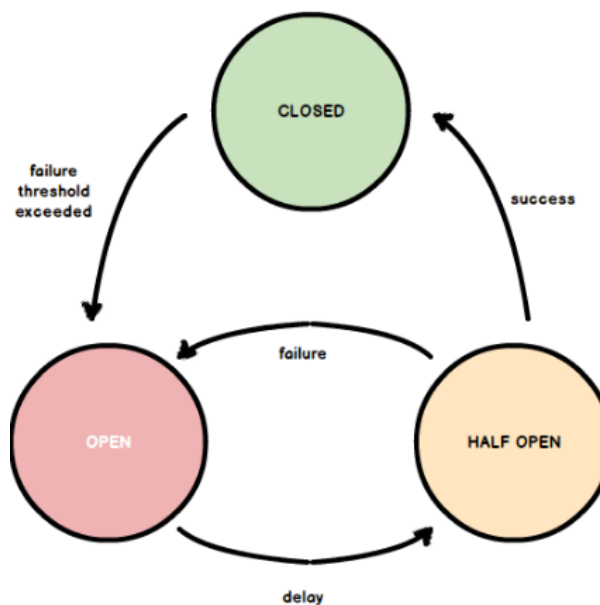
A **circuit breaker** is a design pattern used in microservices and distributed systems to improve reliability. It acts like a safety switch that stops requests to a failing service after a certain number of errors, preventing system overload and long wait times. After a cooldown period, it tests the service again to see if it has recovered, allowing requests to resume. This helps maintain overall system stability and resilience.

Resilience4j is a lightweight, easy-to-use fault tolerance library for Java applications. It

helps developers build resilient systems by providing common patterns like **circuit breakers**, **rate limiters**, **retry mechanisms**, and **bulkheads**. Resilience4j is designed to work well with modern Java frameworks like Spring Boot, allowing you to protect your microservices from failures and improve overall system stability.

We will apply the circuit breaker pattern across all microservices.

These are the 3 possible states for the circuit breaker



- **Closed** ✓

All requests are allowed. If the failure rate exceeds the threshold (e.g., 50%) based on the last N calls, the state transitions to **Open**.

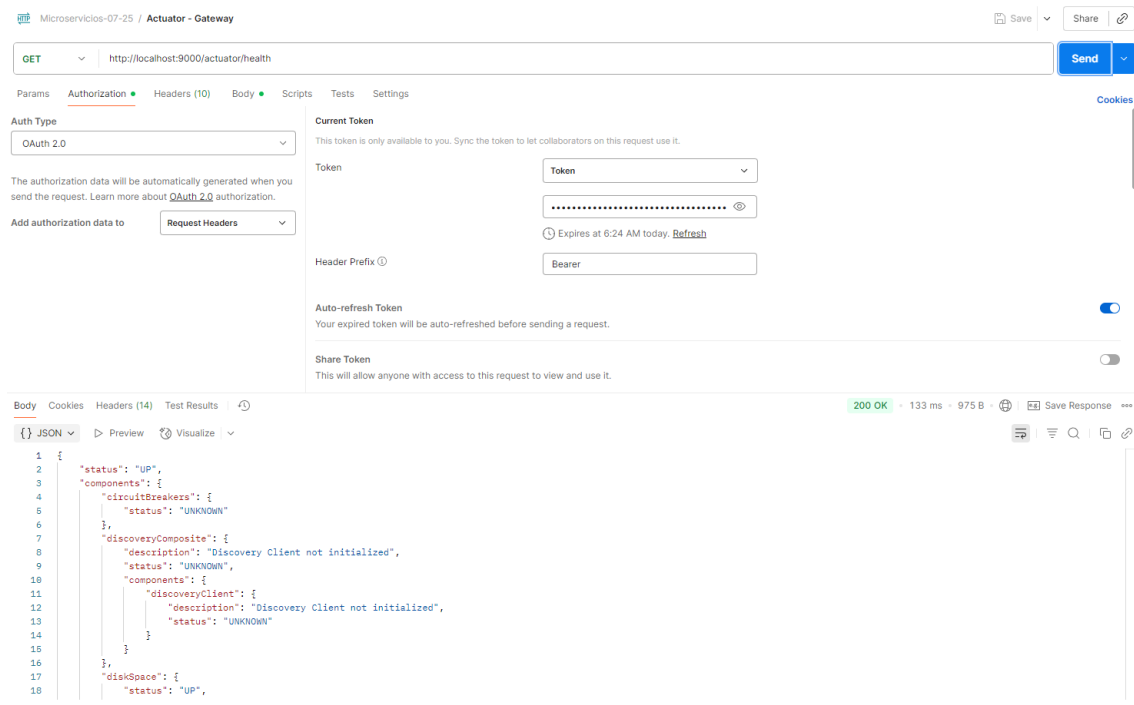
- **Open** ✗

No requests are allowed. The Circuit Breaker waits for a configured **cooldown period** (`waitDurationInOpenState`). After that, it moves to **Half-Open**.

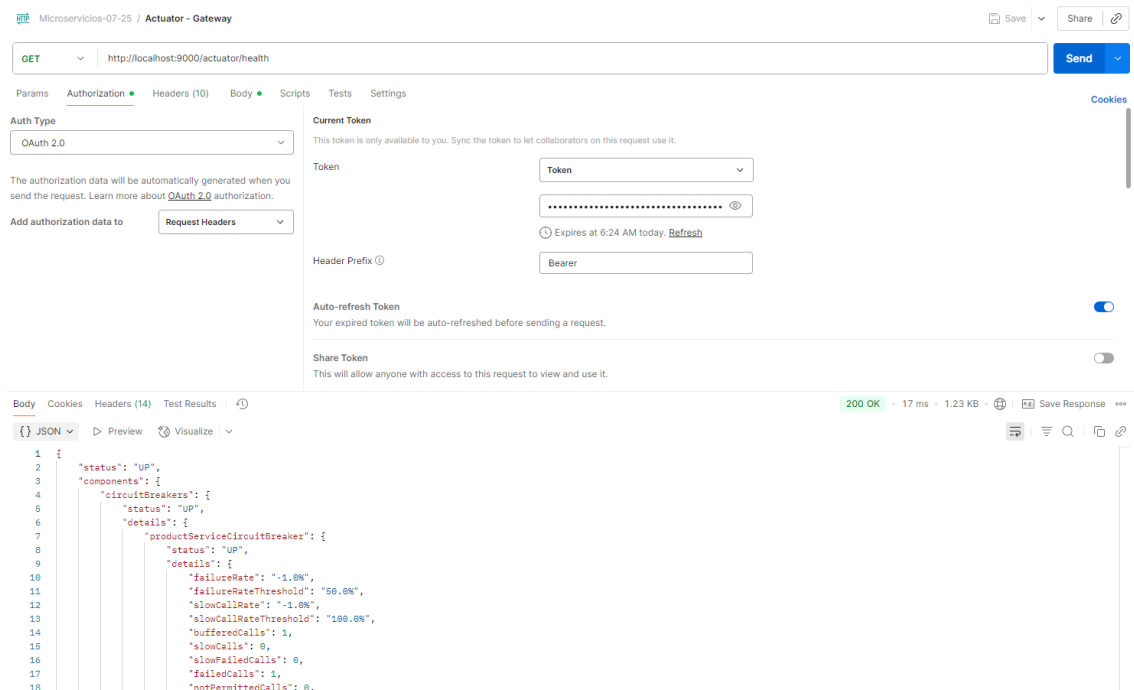
- **Half-Open** ⚠

A limited number of test requests are allowed:

- If they succeed → the breaker **closes** (healthy again).
- If any fail → it goes back to **Open**.



If we stop one of our services, and try to send a request, the circuit breaker will update its status to “UP”, and we will be able to see it through the actuator endpoint



After enabling the actuator and the circuit breaker, we can monitor various metrics through the actuator endpoint, including the circuit breaker status.

Since we configured the circuit breaker to require a minimum of 5 failed requests before changing its status to OPEN, if we force 5 requests while the service is down and then access the actuator endpoint, we can verify that the status is now "OPEN".

```

"circuitBreakers": {
  "status": "UNKNOWN",
  "details": {
    "productServiceCircuitBreaker": {
      "status": "CIRCUIT_OPEN",
      "details": {
        "failureRate": "100.0%",
        "failureRateThreshold": "50.0%",
        "slowCallRate": "0.0%",
        "slowCallRateThreshold": "100.0%",
        "bufferedCalls": 5,
        "slowCalls": 0,
        "slowFailedCalls": 0,
        "failedCalls": 5,
        "notPermittedCalls": 0,
        "state": "OPEN"
      }
    }
  }
}

```

After the configured delay has passed, the status will be set to **HALF_OPEN**.

If **5 valid requests** are sent during this period, the status will be updated to **CLOSED** (it will be updated on the **fifth** valid request).

```

"circuitBreakers": {
  "status": "UNKNOWN",
  "details": {
    "productServiceCircuitBreaker": {
      "status": "CIRCUIT_HALF_OPEN",
      "details": {
        "failureRate": "-1.0%",
        "failureRateThreshold": "50.0%",
        "slowCallRate": "-1.0%",
        "slowCallRateThreshold": "100.0%",
        "bufferedCalls": 0,
        "slowCalls": 0,
        "slowFailedCalls": 0,
        "failedCalls": 0,
        "notPermittedCalls": 0,
        "state": "HALF_OPEN"
      }
    }
  }
}

```

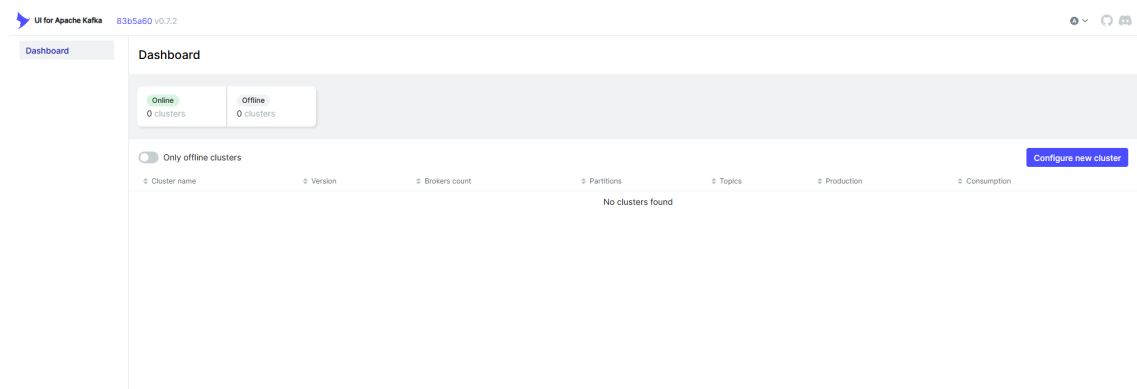
8 - KAFKA

Apache Kafka is an open-source, distributed event streaming platform used for building real-time data pipelines and applications.

It works like a high-performance messaging system where producers send messages (events) to topics, and consumers read those messages.

Kafka is designed for high throughput, scalability, and fault tolerance, making it ideal for microservices communication, log aggregation, and real-time analytics.

Go to localhost:8086 (dockerfile) to access to kafka ui dashboard and configure a new cluster.



Kafka Cluster

Cluster name *

this name will help you recognize the cluster in the application interface

☐ Read-only mode

allows you to run an application in read-only mode for a specific cluster

Bootstrap Servers *

the list of Kafka brokers that you want to connect to



+ Add Bootstrap Server

Truststore

Configure Truststore

Authentication

Configure Authentication

Schema Registry

Configure Schema Registry

Kafka Connect

Configure Kafka Connect

KSQL DB

Configure KSQL DB

Metrics

Configure Metrics

Reset

Validate

Submit

The cluster now appears in the dashboard, and we can access to “Topics”.
If we already made an order, we will see the order-placed topic created.

Dashboard

localhost • ^

Brokers

Topics

Consumers

Topics

🔍 Search by Topic Name

Delete selected topics

Copy selected topic

Purge messages of selected topics

☐	↕ Topic Name	↕ Partitions	↕ Out o
☐	IN __consumer_offsets	50	0
☐	IN _schemas	1	0
☐	order-placed	1	0

We can click on the Messages tab and check the message and its preview

Topics / order-placed

Overview Messages Consumers Settings Statistics

Seek Type

Offset

▼

Offset

Partitions

All items are selected.

▼

Key Serde

String

🔍 Search

✕

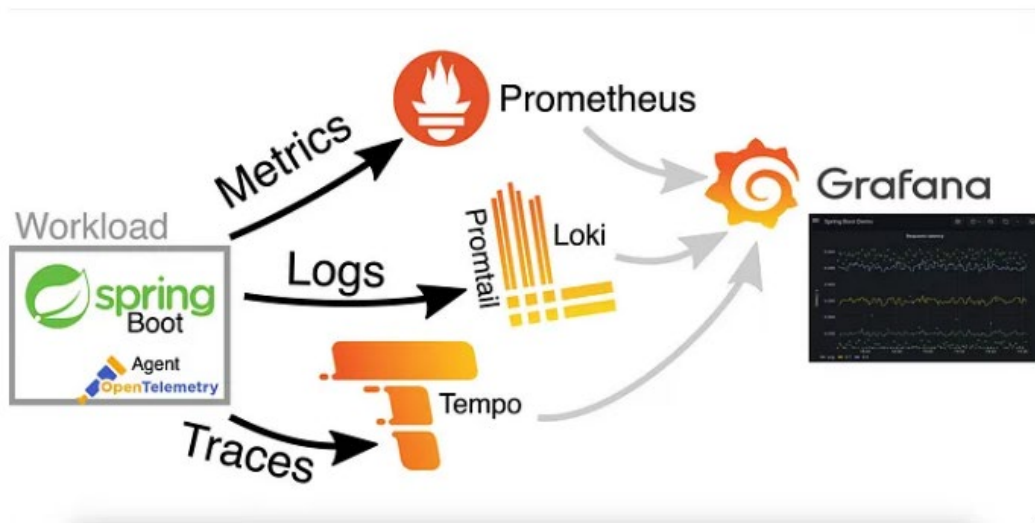
+ Add Filters

	Offset	Partition	Timestamp	Key Preview
+	0	0	13/8/2025, 6:16:16	

← Back

Next →

9 - Grafana stack

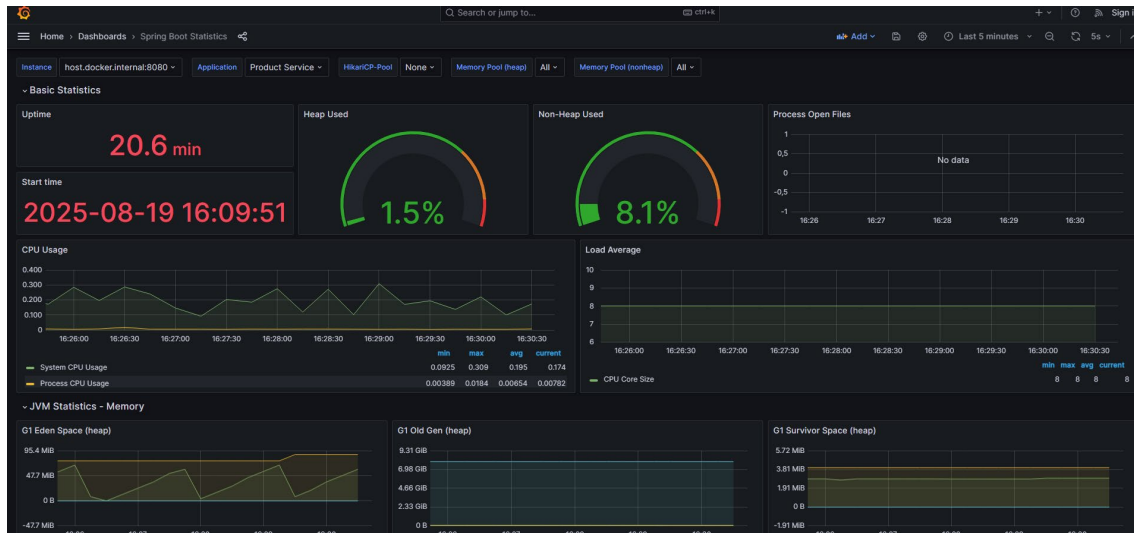


The **Grafana stack** is a set of observability tools for monitoring, logging, and tracing in modern applications. It includes:

- **Prometheus:** A metrics collection and monitoring system that scrapes numeric time-series data from applications and systems.
- **Grafana:** A visualization and dashboard platform that lets you query, visualize, and alert on data from Prometheus and other sources.
- **Loki:** A log aggregation system optimized for storing and querying logs alongside metrics without indexing full log content.
- **Tempo:** A distributed tracing backend that collects traces from applications to help debug and understand request flows.

Together, Grafana, Loki, and Tempo provide **metrics, logging, and tracing** in a single ecosystem, making it easier to gain full observability over modern cloud-native systems.

- **Prometheus** → It collects metrics from your microservices (for example, how many requests each endpoint receives, how long they take, failures, etc.).
- **Grafana** → It's the visual interface where you see those metrics, logs, and traces; basically, it's the tool for monitoring and dashboards.
- **Loki** → It's like a "Prometheus for logs": it stores all the logs from your services and organizes them with labels so you can search and filter them from Grafana.
- **Tempo** → It's for tracing: it stores the traces of requests traveling through your microservices so you can see the full flow of a request and how long each microservice takes.



10 - Kubernetes

Kubernetes (K8s) is an **open-source platform** that automates the deployment, scaling, and management of **containerized applications**.

Why use it?

- Automatically runs and manages containers across many machines.
- Scales apps up or down based on demand.
- Replaces or restarts containers if they fail.
- Provides load balancing and service discovery.

Key concepts

- **Pod:** One or more containers running together.
- **Deployment:** Manages pods and updates.
- **Service:** Gives stable networking to pods.

How it works

1. Build your app into a **Docker image**.
2. Push the image to a **registry**.
3. Deploy it with **Kubernetes manifests**.
4. Kubernetes runs and manages your containers automatically.

Cloud Native Buildpacks

Cloud Native Buildpacks automatically **convert your source code into container images** without writing Dockerfiles.

Key points

- Detects your app's language and dependencies.
- Builds a **ready-to-run container image**.
- Works well with platforms like **Kubernetes** and **Cloud Foundry**.
- Helps **standardize and simplify** container builds for developers.

We can build Docker images using Maven (via the Spring Boot plugin) with the following command:

```
mvn clean spring-boot:build-image -DskipTests
```

kind (Kubernetes IN Docker) is a tool for running local Kubernetes clusters using Docker containers. It is mainly designed for testing and development purposes, allowing you to create lightweight, disposable clusters quickly